



Large data center
data center
data center
data center
data center

Tech Challenge

Tech Challenge é o projeto da fase que englobará os conhecimentos obtidos em todas as disciplinas da fase. Esta é uma atividade que, em princípio, deve ser desenvolvida em grupo. Importante atentar-se ao prazo de entrega, pois trata-se de uma atividade obrigatória, uma vez que vale 60% da nota de todas as disciplinas da fase.

O problema

Após a implantação do sistema inicial para gestão de ordens de serviço, veículos, clientes e controle de peças, a oficina mecânica conquistou maior eficiência no atendimento. Porém, com o aumento da demanda, a expansão para novas unidades e a necessidade de garantir alta disponibilidade do sistema, surgiu a necessidade de evoluir a aplicação. Agora, a oficina busca:

- Reduzir riscos operacionais por meio de infraestrutura escalável;
- Automatizar o provisionamento e o deploy do ambiente;
- Melhorar a qualidade e a organização do código, mantendo a evolução sustentável;
- Preparar a aplicação para suportar grandes volumes de ordens de serviço em horários de pico, com escalabilidade dinâmica.

Objetivo

Evoluir a aplicação desenvolvida na Fase 1 para garantir qualidade, resiliência e escalabilidade, incorporando práticas modernas de infraestrutura e automação.

Requisitos obrigatórios

Evolução da aplicação

- Refatorar o código da fase 1 aplicando:
 - **Clean Code** (nomes claros, simplicidade, coesão).
 - **Clean Architecture ou Arquitetura Hexagonal** (separação adequada de camadas e dependências).
 - Testes automatizados (unitários e/ou integração) para cobrir os fluxos críticos.
- Alterar/criar as seguintes APIs:
 - **Abertura de Ordem de Serviço (OS)**: receber os dados do cliente, veículo, serviços e peças, retornando a identificação única da OS.
 - **Consulta de status da OS**: informar a situação atual da OS (Recebida, Diagnóstico, Aguardando Aprovação, Execução, Finalizada, Entregue).
 - **Aprovação de orçamento**: endpoint para receber notificações externas de aprovação ou recusa do orçamento do cliente.
 - **Listagem de ordens de serviço**:
 - Ordenação por status:
 - Em Execução > Aguardando Aprovação > Diagnóstico > Recebida.
 - Mais antigas primeiro.
 - Excluir (lógica não física) da listagem as OS finalizadas e entregues.
 - **Atualização de status da OS** via alguma ferramenta como e-mail.

Infraestrutura

Containerização:

- Garantir a aplicação containerizada via **Docker**, com:
 - Dockerfile atualizado.

- docker-compose para desenvolvimento local.

Orquestração com Kubernetes (K8s):

- Criar manifestos YAML para deploy em Kubernetes, contemplando:
 - Deployments.
 - Services.
 - ConfigMaps e Secrets (para variáveis sensíveis, como tokens de serviços externos).
 - Horizontal Pod Autoscaler (HPA), escalando conforme consumo de CPU/memória.

Infraestrutura como Código (IaC):

- Criar scripts em **Terraform** para provisionamento do cluster Kubernetes (local ou cloud);
- Banco de Dados;
- Documentar quais recursos estão sendo criados e como aplicar.

Integração Contínua/Entrega Contínua (CI/CD):

- Pipeline de CI/CD configurada (GitHub Actions, GitLab CI, etc.), que execute:
 - Build da aplicação.
 - Execução dos testes automatizados.
 - Build da imagem Docker.
 - Deploy no cluster Kubernetes.
 - Deploy do banco de dados.
 - Aplicação dos manifestos YAML no cluster.

Entregáveis da fase 2

Repositório git (mesmo da fase 1), contendo:

Código-fonte atualizado e refatorado seguindo as boas práticas da abordagem de arquitetura escolhida:

- Dockerfile e docker-compose revisados;
- Manifestos Kubernetes (em /k8s);
- Scripts Terraform (em /infra);
- Arquivos de configuração da pipeline CI/CD.

README.md atualizado com:

- Descrição da solução e dos objetivos desta fase;
- Desenho da arquitetura proposta, incluindo;
 - Componentes da aplicação.
 - Infraestrutura provisionada.
 - Fluxo de deploy.
- Instruções para:
 - Execução local.
 - Deploy em Kubernetes.
 - Provisionamento da infraestrutura com Terraform.
- Link para a collection completa das APIs (Postman, Swagger ou Similar);
- Link para vídeo demonstrativo do ambiente em execução:
 - Publicado no YouTube ou Vimeo (público ou não listado) de até 15 minutos.
 - O vídeo deve demonstrar:
 - Deploy da aplicação.

- Execução do CI/CD.
- Consumo das APIs.
- Escalabilidade automática (pode simular aumento de carga ou múltiplas ordens de serviço).

Entrega no portal do aluno:

PDF contendo o link do repositório **github** compartilhado com o usuário **soat-architecture**; desenho da arquitetura com os recursos escolhidos e link do vídeo (com até 15 minutos de duração) apresentando a solução desenvolvida.